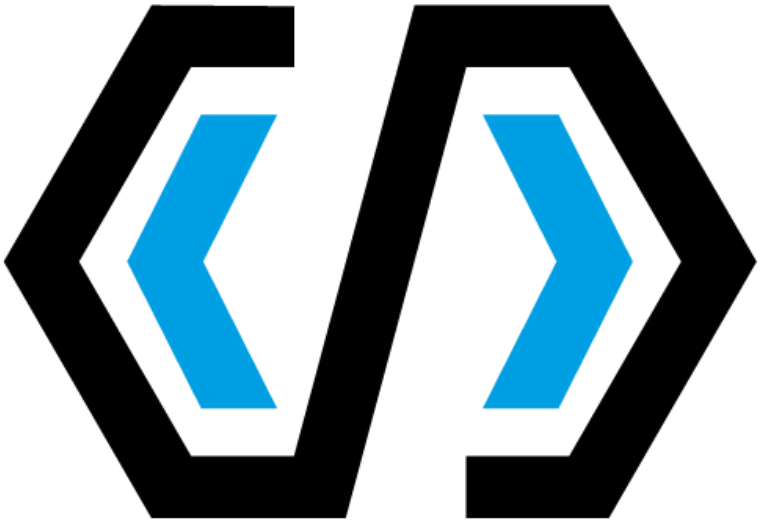
INSTITUTO TÉCNICO RICALDONDE
ESPECIALIDAD DE DESARROLLO DE SOFTWARE



FUNDAMENTOS DE UML: HISTORIA, TIPOS Y CONCEPTOS BÁSICOS

Elaborado por: Msc. Josue A. Guinea Henríquez

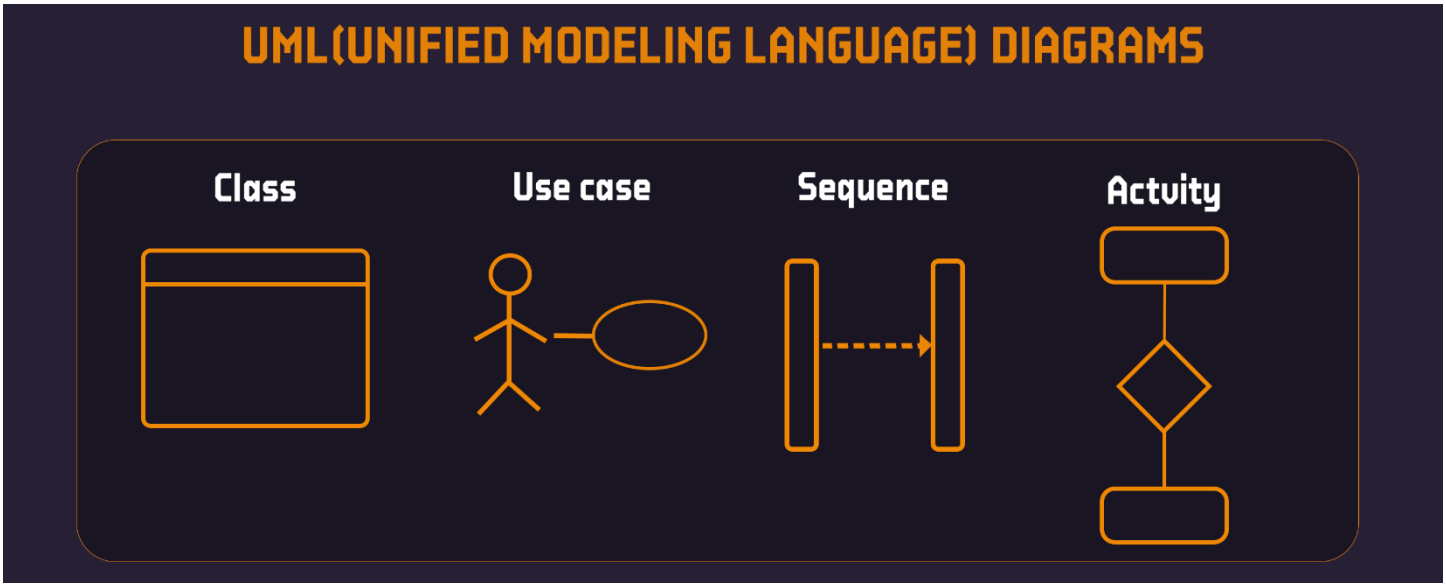
1. Introducción e historia de UML

El **Lenguaje Unificado de Modelado (UML, por sus siglas en inglés: *Unified Modeling Language*)** es un lenguaje gráfico estándar utilizado en la ingeniería de software para visualizar, especificar, construir y documentar los artefactos de un sistema con uso intensivo de software (Booch, Rumbaugh & Jacobson, 2005).

Resulta importante destacar que UML ofrece un vocabulario y unas reglas específicas para combinar elementos de forma visual. Esto permite a los equipos de desarrollo y a las partes interesadas (*stakeholders*) comprender la estructura y el comportamiento de un sistema de manera inequívoca, independientemente del lenguaje de programación que se seleccione para su implementación final.

Figura 1

Diagramas de ejemplo UML



Fuente: Codefinity.com

1.1 La crisis del software y los métodos previos

Para comprender la necesidad del desarrollo de UML, es fundamental revisar el contexto de la ingeniería de software entre las décadas de 1980 y 1990. Durante este período, la industria enfrentaba la creciente complejidad de los sistemas informáticos. El paradigma orientado a objetos experimentaba un aumento en popularidad, pero trajo consigo una problemática conocida en la literatura como la **"Guerra de los Métodos"** (*Method Wars*).

Existían más de 50 metodologías y lenguajes de modelado orientados a objetos, cada uno con su propia notación gráfica y terminología (Fowler, 2003). Entre los más destacados se encontraban:

- **OMT (Object Modeling Technique):** Desarrollado por James Rumbaugh, el cual presentaba fortalezas notables en el análisis y modelado de datos.
- **Método Booch:** Creado por Grady Booch, destacaba principalmente en el diseño y la construcción de la arquitectura del software.
- **OOSE (Object-Oriented Software Engineering):** Ideado por Ivar Jacobson, revolucionó la fase de análisis con la introducción del concepto de los "Casos de Uso".

La falta de un estándar dificultaba la comunicación entre los distintos equipos de trabajo, limitaba la compatibilidad de las herramientas de software y encarecía significativamente la capacitación de los profesionales.

1.2. Evolución del estándar: UML 1.x vs. UML 2.x

El lenguaje ha atravesado múltiples revisiones para adaptarse a las demandas cambiantes de la industria tecnológica:

- **UML 1.x (1997 - 2003):** Se centró en la unificación básica de conceptos. Si bien resolvió la "Guerra de los Métodos", la especificación interna resultaba compleja, presentaba ambigüedades semánticas y carecía de herramientas sólidas para modelar sistemas a gran escala o arquitecturas de componentes físicos.
- **UML 2.x (2005 - Presente):** Representó una reestructuración masiva. La versión UML 2.0 introdujo una base matemática y semántica mucho más rigurosa mediante el uso del metamodelado. Asimismo, aumentó el número de diagramas oficiales y mejoró significativamente la capacidad para representar arquitecturas de software complejas, procesos de negocio y sistemas en tiempo real (Larman, 2004).

1.3. Mitos y realidades

Para la correcta aplicación de UML en entornos profesionales, resulta necesario desmentir algunos conceptos erróneos frecuentes en la industria:

UML es exclusivamente un lenguaje visual. No establece en qué orden se deben ejecutar las tareas ni cómo gestionar un proyecto de software. Se puede implementar de manera complementaria tanto con metodologías ágiles (como Scrum) como tradicionales (como el modelo en Cascada).

Mito: Se requiere utilizar todos los diagramas de UML en un mismo proyecto. **Realidad:** En la práctica, rara vez se emplean todos los diagramas disponibles. El principio fundamental consiste en aplicar únicamente aquellos diagramas que aporten valor y claridad para resolver los problemas específicos del proyecto en cuestión (Fowler, 2003).

Mito: *UML es equivalente al código fuente.* **Realidad:** Aunque existen herramientas que permiten generar código a partir de modelos UML (ingeniería hacia adelante) y viceversa (ingeniería inversa), el lenguaje

opera en un nivel de abstracción superior. Su propósito principal es facilitar el análisis y el diseño conceptual de la solución antes o durante la fase de codificación.

2. Elementos, relaciones, diagramas y mecanismos

El Lenguaje Unificado de Modelado (UML) se fundamenta en un marco de trabajo sólido compuesto por cuatro bloques de construcción esenciales. La comprensión de estos bloques resulta indispensable para modelar sistemas orientados a objetos de manera precisa y estandarizada.

A continuación, se detalla la clasificación teórica de los elementos, las relaciones, los diagramas y los mecanismos comunes que estructuran el lenguaje.

2.1 Elementos de UML

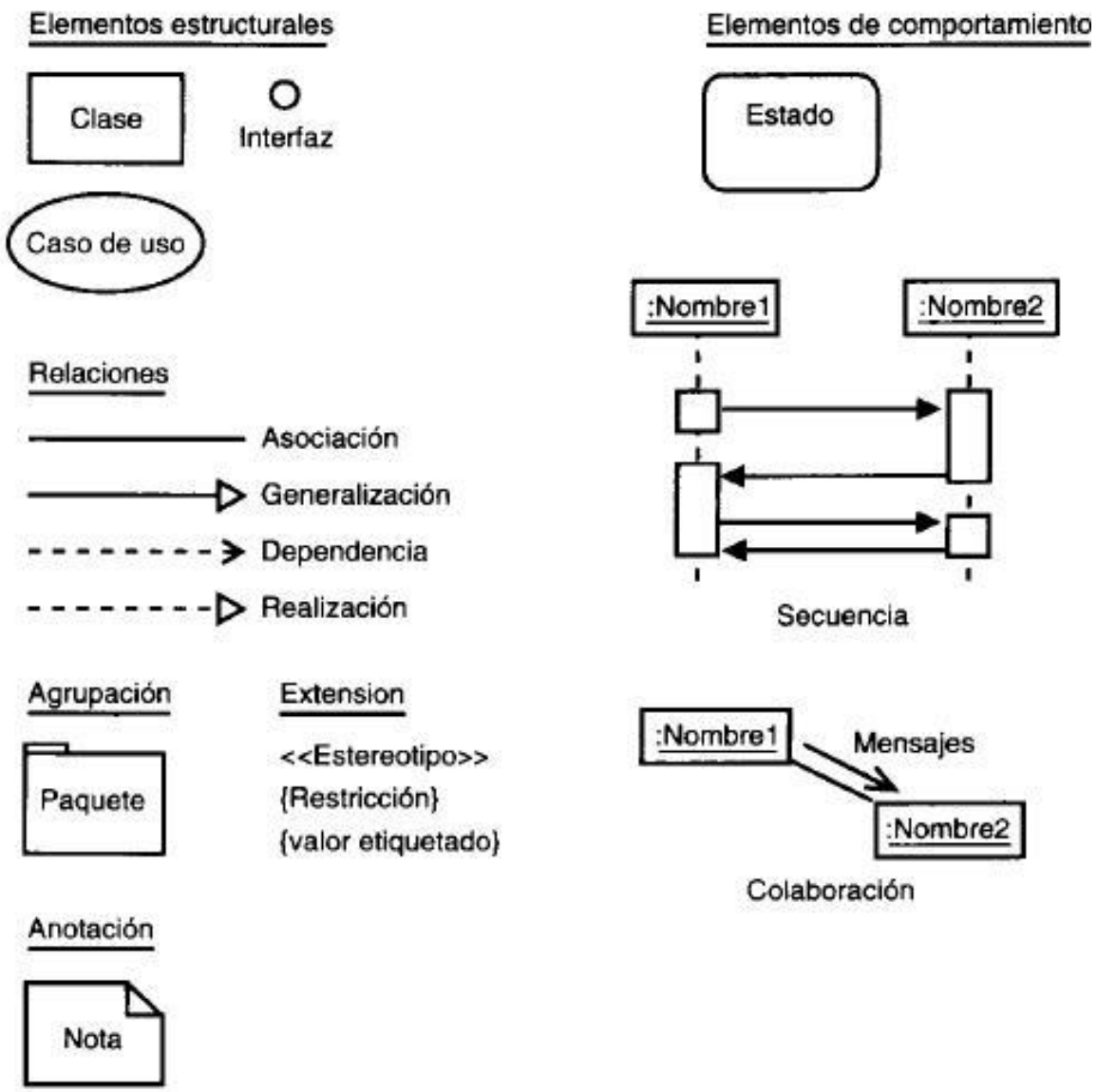
Los elementos constituyen las unidades fundamentales o "ladrillos" con los que se construyen los modelos UML. Se dividen formalmente en cuatro categorías principales según su propósito:

- **Elementos Estructurales:** Representan las partes estáticas de un modelo; son los sustantivos del sistema.
    - *Clases:* Descripciones de un conjunto de objetos que comparten atributos, operaciones, relaciones y semántica.
    - *Interfaces:* Colecciones de operaciones que especifican un servicio de una clase o componente (el contrato).
    - *Casos de Uso:* Descripciones de un conjunto de acciones que el sistema ejecuta para producir un resultado observable.
    - *Componentes y Nodos:* Elementos físicos, donde los componentes representan partes modulares del sistema y los nodos representan elementos de hardware.
- **Elementos de Comportamiento:** Constituyen las partes dinámicas de los modelos; son los verbos del sistema. Representan el comportamiento en el tiempo y el espacio.
    - *Interacciones:* Comportamientos compuestos por un conjunto de mensajes intercambiados entre objetos dentro de un contexto particular.
    - *Máquinas de Estados:* Comportamientos que especifican las secuencias de estados por las que pasa un objeto en respuesta a eventos.
- **Elementos de Agrupación:** Son las partes organizativas de los modelos. El elemento principal es el **Paquete**, el cual sirve para organizar elementos estructurales, de comportamiento e incluso otros paquetes en un sistema manejable.

- **Elementos de Anotación:** Son las partes explicativas de los modelos. Se materializan principalmente como **Notas** (comentarios), las cuales se adjuntan a un elemento o conjunto de elementos para incluir restricciones, explicaciones o información textual adicional.

Figura 2

Elementos de UML



Fuente: maritzacondori.wordpress.com

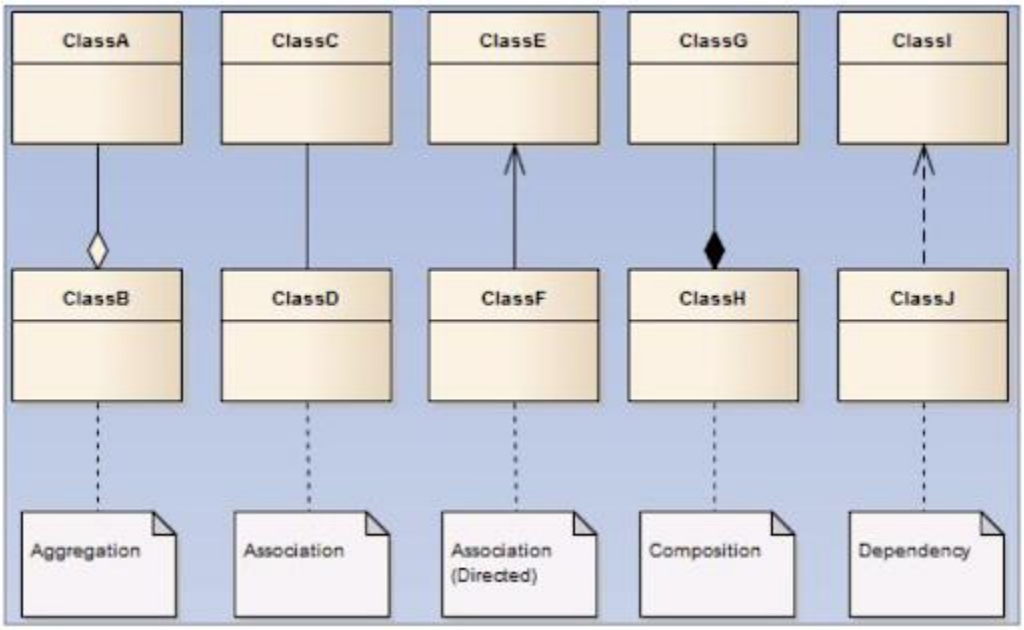
2.2. Relaciones en UML

Las relaciones son los enlaces lógicos que unen a los diferentes elementos entre sí para dotar de semántica al modelo general. Existen cuatro tipos fundamentales de relaciones:

- **Dependencia:** Es una relación semántica entre dos elementos donde un cambio en el elemento independiente (proveedor) puede afectar la semántica del elemento dependiente (cliente). Se representa con una línea discontinua con una flecha abierta.
- **Asociación:** Es una relación estructural que describe una conexión entre objetos. Se representa con una línea continua. Incluye dos variaciones semánticas más fuertes que indican relaciones de tipo "todo-parte":
    - *Agregación:* El ciclo de vida de la parte es independiente del ciclo de vida del contenedor.
    - *Composición:* El ciclo de vida de la parte depende estrictamente del ciclo de vida del contenedor.
- **Generalización:** Es una relación taxonómica o jerárquica entre un elemento general (superclase) y un elemento más específico (subclase). Indica que el elemento específico hereda las características del general. Se ilustra con una línea continua con una punta de flecha vacía y cerrada.
- **Realización:** Es una relación semántica entre clasificadores, donde un clasificador especifica un contrato que otro clasificador garantiza llevar a cabo. Resulta común entre las interfaces y las clases que las implementan. Se traza con una línea discontinua y una punta de flecha vacía y cerrada.

Figura 3

Relaciones de UML



Fuente: blog.visual-paradigm.com

2.3. Diagramas de UML

Los diagramas son las representaciones gráficas de un conjunto de elementos y relaciones. En la versión actual de UML (2.x), los diagramas se agrupan en dos grandes dimensiones para abarcar todas las perspectivas del ciclo de vida del software:

- **Diagramas Estructurales:** Enfatizan los elementos que deben existir en el sistema modelado. Modelan la vista estática e incluyen, entre otros, el Diagrama de Clases, Diagrama de Componentes, Diagrama de Objetos y Diagrama de Despliegue.
- **Diagramas de Comportamiento:** Enfatizan lo que debe suceder en el sistema modelado. Modelan la vista dinámica e incluyen el Diagrama de Casos de Uso, Diagrama de Actividades y Diagrama de Máquina de Estados.
    - *Nota:* Los **Diagramas de Interacción** (Secuencia, Comunicación, Tiempos) son un subconjunto de los diagramas de comportamiento enfocados específicamente en el flujo de control y datos entre los elementos que participan.

3. Referencias bibliográficas

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2ª ed.). Addison-Wesley Professional.

Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3ª ed.). Addison-Wesley Professional.

Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3ª ed.). Prentice Hall.

Object Management Group (OMG). (2017). *About the Unified Modeling Language Specification Version 2.5.1*. Recuperado de la documentación oficial de OMG.